LAMP: A Selective-ACK Transport Protocol for Reliable Bulk Data Transmission over LoRa Networks

Alessandro Monticelli

***Abstract*—This paper presents Long-Range Adaptive Multi-Packet Protocol (LAMP), a lightweight transport architecture that enables the continuous transfer of large data blocks over point-to-point Long Range (LoRa©) networks. To overcome the severe Time-on-Air overhead and duty-cycle exhaustion typical of existing Stop-and-Wait solutions, the proposed architecture segments payloads into up to 255 chunks per transmission window. Each chunk is secured by a compact 9-byte Logical Header incorporating IPv6-ready node addressing and a Cyclic Redundancy Check (CRC) checksum. The protocol provides four distinct operational modes combining connection state (Connectionless / Stateful) and reliability (Best-effort / SACK-ARQ feedback). By requiring receiver feedback only at sequence boundaries rather than per-packet, the Selective Acknowledgment (SACK) mechanism significantly reduces acknowledgment overhead without relying on heavy Forward Error Correction (FEC).**

***Index Terms*—LoRa©, SACK, packet fragmentation, transport protocol, Internet of Things (IoT), embedded systems, 3-Way Handshake, point-to-point.**

I. Introduction

LOW-POWER, low-cost, and long-range wireless communication technologies are becoming increasingly important in the context of the IoT. Applications such as environmental monitoring, precision agriculture, and autonomous Unmanned Aerial Vehicles (UAVs) require reliable data transmission over extended distances under strict power and hardware constraints [1].

Several communication standards, including ZigBee, Sigfox, and LoRa© have been adopted to address these needs, and LoRa©, in particular, has emerged as the leading choice for low-power and long-range deployments thanks to its robust modulation technique and open network architecture [2]. However, conventional solutions based on LoRa© (particularly those following the Long Range Wide Area Network (LoRaWAN©) specifications) are optimized for small, non-frequent payloads [3].

This limitation poses severe restrictions on data-intensive IoT applications—such as firmware-over-the-air (FOTA) updates, Simultaneous Localization and Mapping (SLAM) map transfers in autonomous robotics, or bulk sensor log collection—which demand the continuous transfer of large data blocks [4], [5]. Standard LoRaWAN© links require substantial transmission overhead, restrictive duty-cycle limitations [6], and lack optimized mechanisms for handling large, multi-packet payloads in point-to-point configurations.

This paper presents the design and implementation of LAMP, a lightweight transport protocol for point-to-point LoRa© networks. While early iterations focused on payload segmentation, successive additions—a SACK scheme, a hardware abstraction layer, and dual reliable/best-effort transmission modes—have evolved the architecture into a complete transport solution supporting payloads up to 65.0 KB (or 62.2 KB on 255-byte-First-In, First-Out (FIFO) transceivers such as the Semtech SX1262), operating independently of higher-layer middleware such as LoRaWAN©.

The main contributions of this work are:

- **Bitmap-Based SACK Scheme**: Unlike existing Low-Power Wide-Area Network (LPWAN) fragmentation solutions that rely on Stop-and-Wait (S&W) Automatic Repeat Request (ARQ)—which requires one acknowledgment per packet and incurs severe latency under duty-cycle constraints—the proposed protocol uses a compact binary bitmap carried in a single feedback frame. The receiver reports the delivery status of up to 255 chunks per round-trip, reducing retransmission overhead and over-the-air footprint.
- **Lightweight Packet Segmentation and Abstraction**: The protocol introduces a 9-byte logical header (with 8-bit IPv6-ready node addressing) and a custom CRC integrity check on partial payloads, allowing robust reassembly of out-of-order chunks, even on resource-constrained microcontrollers.
- **Modular C++17 Implementation and Test-Driven Development (TDD) Validation**: The core state machine is completely decoupled from the physical radio driver through a Hardware Abstraction Layer (HAL). This design enables both native host-based unit testing (using the Unity framework) and physical deployment on Semtech SX126x transceivers, bridging the gap between simulation and real-world embedded systems.

II. State of The Art

Reliable packet fragmentation and reassembly over LPWANs has been the subject of active research, driven by the growing demand for data-intensive IoT applications.

A prominent industrial standard is Static Context Header Compression (SCHC) [5], which introduces a framework for fragmenting IPv6 packets over resource-constrained networks. To guarantee reliability, SCHC ACK frames carry a bitmap representing the status of individual tiles within a window

(ACK-on-Error). However, SCHC is designed as a generic adaptation layer for IP-based architectures, requiring pre-shared context rules and gateway-centric networks. Similarly, the LoRa© Alliance standardized the Fragmented Data Block Transport specification to support Firmware Update Over The Air (FUOTA) using FEC codes to reconstruct blocks without feedback. However, FEC-based recovery introduces a constant over-the-air encoding overhead and high computational requirements for low-cost embedded systems.

For custom peer-to-peer LoRa© networks, early efforts focused on overcoming the severe airtime latencies and capacity constraints of traditional Stop-and-Wait (S&W) ARQ schemes. Empirical link evaluations by Cattani et al. [7] and network scalability analyses by Bor et al. [8] demonstrated that high packet-loss rates and strict Time-on-Air limits make per-packet Stop-and-Wait acknowledgments unviable for bulk data transfers [4]. A pioneering contribution in this domain was introduced by Chen et al. [9] with the Multi-Packet LoRa (MPLR) protocol. MPLR introduced the core concept of replacing Stop-and-Wait ARQ with a windowed batch transmission scheme acknowledged by a bit-vector acknowledgment packet (`BVACK`). Subsequent specialized studies explored multi-hop relay networks for visual IoT coverage [10], comprehensive mesh survey frameworks [11], and Time Division Multiplexing (TDM) scheduled channels [12]. It is worth noting that LAMP is engineered as a pure transport-layer protocol; as such, it remains topology-agnostic and can operate transparently on top of multi-hop mesh routing protocols to extend reliable bulk delivery across multi-hop topologies.

While LAMP draws foundational inspiration from the bit-vector SACK mechanism introduced in MPLR [9], our architecture introduces major engineering and protocol-level advancements that generalize and optimize the concept for modern embedded IoT applications:

- **Header and Protocol Overhead Optimization**: MPLR relies on a 16-byte header per fragment (carrying 4-byte Destination EUI, 4-byte Source EUI, Service Number, Sequence Number, Flags, Payload Size, Batch Size, and Checksum). In contrast, LAMP utilizes a 9-byte Logical Header (1-byte `srcAddr`, 1-byte `dstAddr`, 2-byte `messageId`, 1-byte `totalChunks`, 1-byte `chunkIndex`, 1-byte `payloadSize`, 1-byte `flags`, and 1-byte `protocolVersion`). Including the 2-byte CRC-16 checksum suffix, LAMP requires a total over-the-air overhead of 11 bytes per frame compared to MPLR's 16 bytes, yielding a 5-byte reduction in total over-the-air overhead per fragment.
- **Target Scope**: MPLR was engineered specifically as an application-level image transmission protocol for agricultural uplink nodes to a central gateway. LAMP is designed as a domain-agnostic, general-purpose transport layer capable of segmenting any arbitrary data payload up to 62.2 KB on standard LoRa© transceivers (ranging from high-frequency sensor telemetry arrays and robotic SLAM keyframes to firmware binaries and multiagent state matrices).
- **Flexible Operational Modes**: MPLR operates under a single stateful batch delivery model. In contrast, LAMP provides four operational modes spanning *Unreliable Connectionless* (best-effort datagrams), *Unreliable Connected* (session-tracked streaming), *Reliable Connectionless* (stateless SACK-ARQ), and *Reliable Connected* (stateful negotiated streams).
- **Dynamic Parameter Negotiation**: MPLR utilizes a fixed batch size and requires a 4-way handshake tear-down (`FIN`/`ACK`). LAMP implements a lightweight 3-Way Handshake (3WHS) embedded with dynamic parameter negotiation (`SynMetadata` and `SynAckMetadata` negotiating chunk size, sliding window, and inactivity timeout) and replaces explicit tear-down overhead with automatic idle session pruning.

To clearly position the proposed protocol within the current technological landscape, Table I provides an architectural comparison with other prominent fragmentation and transport solutions over LoRa©, including MPLR.

III. Proposed System Architecture

LAMP is a transport-layer protocol running directly over the LoRa© physical layer. It segments arbitrarily large payloads at the transmitter and reconstructs them at the receiver while minimizing Logical Header and Over-the-Air (OTA) overhead.

A. Protocol Specification and Design Iterations

The development of LAMP followed an iterative trajectory, progressing from an initial stateless baseline (LAMPv1) to a stateful, addressed transport architecture (LAMPv2).

1) Stateless Header Specification: The initial protocol version, LAMPv1, was engineered for maximum header compression during periodic sensor telemetry transfers over point-to-point links. It utilized a compact 7-byte Logical Header containing only sequence tracking and payload bounds (`messageId` [2 B], `totalChunks` [1 B], `chunkIndex` [1 B], `payloadSize` [1 B], `flags` [1 B], and `protocolVersion` [1 B]).

While LAMPv1 achieved high spectral efficiency for isolated datagram bursts, real-world deployment on embedded hardware exposed three critical architectural vulnerabilities:

- *Speculative Buffer Allocation*: Receivers were forced to allocate reassembly memory speculatively upon receiving the first fragment of an unannounced stream. When receiving large payloads (e.g., image tiles or firmware blocks), memory-constrained microcontrollers frequently suffered heap exhaustion or buffer overflows.
- *Session Interference*: Lacking explicit source and destination identifiers, multiple unaddressed transmitters sharing the same RF channel risked interleaving fragments with identical sequence numbers, leading to corrupted reassembly buffers.
- *Static Transport Limits*: Fixed chunk sizes and static timeout thresholds prevented dynamic adaptation to link quality degradation or variable receiver memory limits.

2) Addressed Logical Header Specification: To resolve the limitations of LAMPv1, LAMPv2 introduces an addressed, stateful transport architecture. LAMPv2 expands the Logical Header to 9 bytes by embedding explicit 8-bit Source

TABLE I
ARCHITECTURAL COMPARISON OF LORA© TRANSPORT SOLUTIONS

Feature	Stop-and-Wait	SCHC [5]	LoRaWAN© Frag.	MPLR [9]	Proposed (LAMP)
ACK / Retransmission	Per-Packet ACK	ACK-on-Error Bitmap	None (FEC Code)	Bit-Vector SACK	Bit-Vector SACK
Header Size	2–4 Bytes	1–2 Bytes	2 Bytes	16 Bytes	9 Bytes (11B total w/ CRC)
Max Frame Payload	240+ Bytes	Variable	Variable	239 Bytes	244 Bytes
Operational Modes	1 (Reliable S&W)	2 (ACK / No-ACK)	1 (Unreliable FEC)	1 (Reliable Batch)	4 Operational Modes
Handshake / Setup	None	Static Rules	Session Setup	3WHS + 4WHS Teardown	3WHS Dynamic Negotiation
Target Scope	Simple P2P	IP Star Network	Gateway FUOTA	Image Upload	General-Purpose P2P / Multi-Node

(`srcAddr`) and Destination (`dstAddr`) Node Identifiers, while incorporating an integrated 3WHS mechanism.

Although adding 2 bytes to the header increases over-the-air frame length by less than 0.8% on a 244-byte payload, this trade-off delivers essential architectural guarantees:

1) *Buffer Memory Safety*: The 3WHS exchange allows nodes to negotiate reassembly buffer limits (`SynMetadata`) prior to payload transmission, preventing allocation failures on memory-constrained receivers.
2) *Channel Session Isolation*: Explicit node addressing completely prevents promiscuous packet hijacking and session corruption in multi-node wireless topologies.
3) *IPv6 Addressing Compatibility*: The 8-bit address space maps natively to local IPv6 link-local subnet suffixes without the overhead of full 128-bit IP headers over the air.

B. Operational Modes and Sequence Management

To accommodate diverse IoT workload requirements—ranging from ultra-fast periodic telemetry streams to critical high-capacity firmware or imaging transfers—LAMP defines four distinct operational modes across two primary dimensions: *Reliability* (Unreliable Best-Effort vs. SACK-ARQ) and *Connection State* (Connectionless Stateless vs. Connection-Oriented Stateful).

1) Implicit Frame Boundary Derivation: To maintain a compact 9-byte header while supporting robust out-of-order packet reassembly, LAMP avoids dedicated boundary control bits. Instead, Start-of-Message (SOM) and End-of-Message (EOM) boundaries are implicitly derived at the receiver directly from the 0-based fragment index (`chunkIndex`) and the total fragment count (`totalChunks`):

$$\text{SOM} \iff \texttt{chunkIndex} == 0 \tag{1}$$

$$\text{EOM} \iff \texttt{chunkIndex} == \texttt{totalChunks} - 1 \tag{2}$$

Explicitly carrying `totalChunks` in every fragment header allows the receiver to instantly determine the exact message bounds and pre-allocate the required reassembly buffer and SACK bitmap upon receiving *any* fragment, even if fragments arrive out of order or if the initial chunk is lost.

2) Node Addressing Scheme: To support multi-node communication without incurring the prohibitive airtime overhead of full 128-bit IPv6 headers on LoRa© PHY payloads, LAMP introduces a compact 8-bit Node Identifier space:

- **`0x00` (`ADDRESS_UNASSIGNED`)**: Reserved for legacy or unaddressed P2P datagrams (promiscuous mode).
- **`0x01–0xFE`**: 254 unique individual node addresses (Unicast).
- **`0xFF` (`ADDRESS_BROADCAST`)**: Broadcast address processed by all active receiving nodes on the RF channel.

This 8-bit Node ID design is IPv6-ready: at an edge gateway or application layer, a local 8-bit Node ID (e.g., `0x42`) maps directly to an IPv6 link-local subnet suffix (e.g., `fe80::42`), enabling direct IP-layer mapping without transmitting heavy IPv6 headers over the constrained radio link.

The 8-bit address space supports up to 254 unicast nodes per RF cluster, which is sufficient for the dense but bounded deployments targeted by this work (e.g., agricultural sensor arrays, UAV swarm sub-clusters). For applications requiring broader network reach, Section VII foresees integration of full IPv6 routing, where the 8-bit node identifier serves as a local cluster suffix rather than a global unique identifier.

The exact byte alignment and offsets of the 9-byte header packet structure are presented in Table III and Figure 1.

The fields in the Logical Header are defined as follows:

- **`srcAddr` (1 byte)**: Source node identifier (`0x01–0xFE`, or `0x00` for unassigned).
- **`dstAddr` (1 byte)**: Destination node identifier (`0x01–0xFE`, or `0xFF` for broadcast).
- **`messageId` (2 bytes)**: Unique message sequence identifier. All fragments composing the same segmented message share this ID.
- **`totalChunks` (1 byte)**: The total count of chunks into which the original large payload is divided ($1 \leq \texttt{totalChunks} \leq 255$).
- **`chunkIndex` (1 byte)**: The 0-based index of this specific fragment ($0 \leq \texttt{chunkIndex} < \texttt{totalChunks}$).
- **`payloadSize` (1 byte)**: Payload data byte count. Logically, the 1-byte field supports up to 255 bytes per chunk, enabling a theoretical protocol capacity of $255 \times 255 = 65,025$ bytes (≈ 65.0 KB) per session.

TABLE II
LAMP OPERATIONAL MODES ARCHITECTURE

Operational Mode	Reliability & Connection State	Primary Target Use Case
Mode 1: Best-Effort Datagram	Unreliable, Connectionless (Stateless)	High-frequency telemetry streams, periodic sensor data bursts.
Mode 2: Best-Effort Stream	Unreliable, Connection-Oriented (Stateful)	Session-tracked real-time sensor streams without retransmission delays.
Mode 3: SACK-Reliable Datagram	Reliable (SACK), Connectionless (Stateless)	Isolated high-priority sensor bursts requiring delivery confirmation.
Mode 4: SACK-Reliable Stream	Reliable (SACK), Connection-Oriented (3WHS)	Critical large payload transfers (FUOTA binaries, camera images, SLAM maps).

Byte 0	Byte 1	Byte 2 – 3	Byte 4	Byte 5	Byte 6	Byte 7	Byte 8
`srcAddr`	`dstAddr`	`messageId`	`totalChunks`	`chunkIndex`	`payloadSize`	`flags`	`protocolVersion`

Fig. 1. Byte layout of the compact 9-byte Logical Header incorporating Node Addressing.

TABLE III
LAMP OTA PACKET LAYOUT AND BYTE OFFSETS

Offset	Field Name	Type	Description
0	`srcAddr`	uint8_t	Source Node ID (`0x01`–`0xFE`)
1	`dstAddr`	uint8_t	Destination Node ID (`0x01`–`0xFE`, `0xFF`)
2–3	`messageId`	uint16_t	Message Sequence ID
4	`totalChunks`	uint8_t	Total Count ($1 \leq N \leq 255$)
5	`chunkIndex`	uint8_t	Index ($0 \leq i <$ `totalChunks`)
6	`payloadSize`	uint8_t	Payload Count ($N \leq 244$ B)
7	`flags`	uint8_t	Bitfield (`ACK_REQ`, `CONN_REQ`)
8	`protocolVersion`	uint8_t	Protocol Version (`0x02`)
9–N+8	`payload[]`	uint8_t[]	Application Data (N bytes)
N+9–N+10	`crc`	uint16_t	CRC-16 Checksum

On standard transceivers with a 255-byte hardware FIFO limit (e.g., SX1262), subtracting the 9-byte header and 2-byte CRC-16 yields $N = 244$ bytes per chunk, resulting in an empirical capacity of $255 \times 244 = 62,220$ bytes (≈ 62.2 KB).

- **`flags` (1 byte)**: A control bitfield used to manage transmission and connection states:
  - `FLAG_ACK_REQ` (`0x04`): Selective Acknowledgment requested from receiver.
  - `FLAG_ACK` (`0x08`): SACK feedback packet containing binary bitmap of received chunks.
  - `FLAG_CONN_REQ` (`0x10`): Connection Request control frame.
  - `FLAG_CONN_ACK` (`0x20`): Connection Acknowledged control frame.
  - `FLAG_CONN_NACK` (`0x40`): Connection Refused control frame.
- **`protocolVersion` (1 byte)**: Identifies protocol version (currently 2) to ensure backward compatibility.

After the payload, a 16-bit CRC-16 (Modbus CRC-16, polynomial `0xA001`, initialized to `0xFFFF`) is appended. The checksum is computed over the concatenation of the 9-byte Logical Header and exactly `payloadSize` application-layer bytes; any trailing buffer bytes beyond `payloadSize` are excluded, so that partial final chunks are validated identically to full-size chunks and inter-implementation interoperability is preserved.

Table IV summarises all defined flag combinations and the behaviour of the packet parser upon receipt of an undefined value.

TABLE IV
LAMP FLAG FIELD: VALID COMBINATIONS AND PARSER BEHAVIOUR

Value	Semantic	Notes
`0x00`	Data chunk	Unreliable/reliable chunk, no ACK
`0x04`	Final chunk (ACK_REQ)	Receiver replies with SACK
`0x08`	SACK feedback	Payload = received-chunk bitmap
`0x10`	SYN	3WHS Step 1 (SynMetadata)
`0x30`	SYN-ACK	3WHS Step 2 (SynAckMetadata)
`0x20`	ACK	3WHS Step 3, no payload
`0x40`	CONN-NACK	Payload = rejection reason code
other	*Undefined*	Silently discarded

C. Selective Acknowledgment and Retransmission

In reliable mode, the protocol implements a SACK mechanism [13] to recover lost fragments with minimal additional airtime.

1) Retransmission Protocol Interaction: When a transmission is executed in reliable mode:

1) The transmitter segments the payload, serializes the chunks with `srcAddr` and `dstAddr`, and streams them sequentially. The final chunk index ($Index =$ `totalChunks` $- 1$) is marked with the `FLAG_ACK_REQ` flag.
2) Upon receipt of the chunk requesting acknowledgment, the receiver evaluates its reassembly session. It identifies which fragments are still missing by checking its internal bitmask of received chunks.
3) The receiver responds by transmitting a SACK packet (`FLAG_ACK`) addressed back to the transmitter's

`srcAddr`. The SACK's payload contains a binary bitmap where each bit represents the receipt status of a chunk (1 for received, 0 for missing).

4) The transmitter parses the received SACK bitmap. If some chunks are missing, it selectively retransmits only those lost chunks. The last retransmitted chunk has the `FLAG_ACK_REQ` flag set again to query the receiver.
5) This selective retry loop continues until the full message is reconstructed at the receiver, or the transmitter reaches the maximum retry count.

2) Half-Duplex Hardware Switching Guard: Because LoRa© transceivers are half-duplex, a hardware transition period is required for the transmitter to switch from transmit (TX) mode to receive (RX) mode after sending a packet. While physical SX1262 transceiver state transitions take $< 100\,\mu$s, the 25 ms guard delay accounts for software-level Real-Time Operating System (RTOS) task scheduling jitter, Serial Peripheral Interface (SPI) bus transaction queuing, and microcontroller wake-up latencies on embedded targets.

It is important to emphasize that all protocol timing parameters, guard delays (e.g., `SACK_PREAMBLE_GUARD_DELAY_MS`), and retry limits are fully decoupled into a centralized configuration namespace (`LoRaMultiPacketConfig`) and can be customized by integrators for any MCU/radio target. The default parameters presented in this study were specifically tuned around the Heltec Wi-Fi LoRa© 32 V3 (ESP32-S3) hardware platform.

3) Timeout and Fallback Recovery: If the transmitter does not receive a SACK packet within the dynamic timeout window, it assumes the query or response was lost and retransmits the last `FLAG_ACK_REQ`-marked chunk. Up to 5 consecutive timeouts are allowed before the transmission is aborted.

The timeout is computed adaptively as:

$$T_{\text{timeout}} = \max\Big(T_{\min},\ \alpha \cdot \widehat{T}_{\text{oA}} + \delta\Big) \tag{3}$$

where $T_{\min} = 1000$ ms is the minimum guard, $\alpha = 1.5$ is a safety factor, $\widehat{T}_{\text{oA}}$ is the Time-on-Air for the current packet length queried from the radio driver [14], and $\delta = 25$ ms is the half-duplex switching guard delay. Table V reports representative timeout values across spreading factors.

4) Analytical Burst Reliability Bounding: Under a frame loss probability p, the probability that a specific retransmitted chunk or SACK query fails across k maximum retries is p^k. Consequently, for a data payload segmented into N chunks, the overall burst delivery success probability P_{success} and overall burst failure probability $P_{\text{burst\_fail}}$ under SACK-ARQ are analytically bounded by:

$$P_{\text{success}} = \left(1 - p^k\right)^N, \qquad P_{\text{burst\_fail}} = 1 - \left(1 - p^k\right)^N \tag{4}$$

For a lossy wireless channel with a 20% frame error rate ($p = 0.20$) and the default retry limit $k = 5$, the individual chunk failure probability drops to $p^5 = 0.00032$, yielding an overall burst delivery success probability $P_{\text{success}} > 99.993\%$ for a standard 20-chunk payload transfer.

TABLE V
ADAPTIVE SACK TIMEOUT VALUES (BW 500 KHZ, CR 4/5, 253-BYTE FRAME)

SF	$\widehat{T}_{\text{oA}}$ (ms)	$\alpha \cdot \widehat{T}_{\text{oA}} + \delta$ (ms)	T_{timeout} (ms)
7	92	163	1000
8	128	217	1000
9	215	348	1000
10	371	582	1000
11	742	1138	1138
12	1484	2251	2251

D. Memory Bounding and RAM Footprint Analysis

To guarantee memory safety on constrained microcontrollers, the dynamic memory allocated for an active reassembly session at the receiver is strictly bounded by:

$$M_{\text{session}} = S_{\text{struct}} + N_{\text{chunks}} \cdot S_{\text{chunk}} + \left\lceil \frac{N_{\text{chunks}}}{8} \right\rceil \tag{5}$$

where $S_{\text{struct}} \approx 48$ bytes represents the session tracking structure metadata, $S_{\text{chunk}} \leq 244$ bytes is the maximum payload chunk size, and the final term represents the binary SACK bitmask.

For a representative 20-chunk payload (4.88 KB data size), a complete reassembly session requires only ≈ 4.93 KB of RAM. Furthermore, the protocol bounds the maximum number of concurrent active sessions (`MAX_CONCURRENT_SESSIONS = 10`), placing an absolute upper bound of ≈ 49.3 KB on total protocol RAM consumption, making it well-suited for low-cost IoT microcontrollers with limited SRAM capacities.

E. Formal Protocol Algorithms

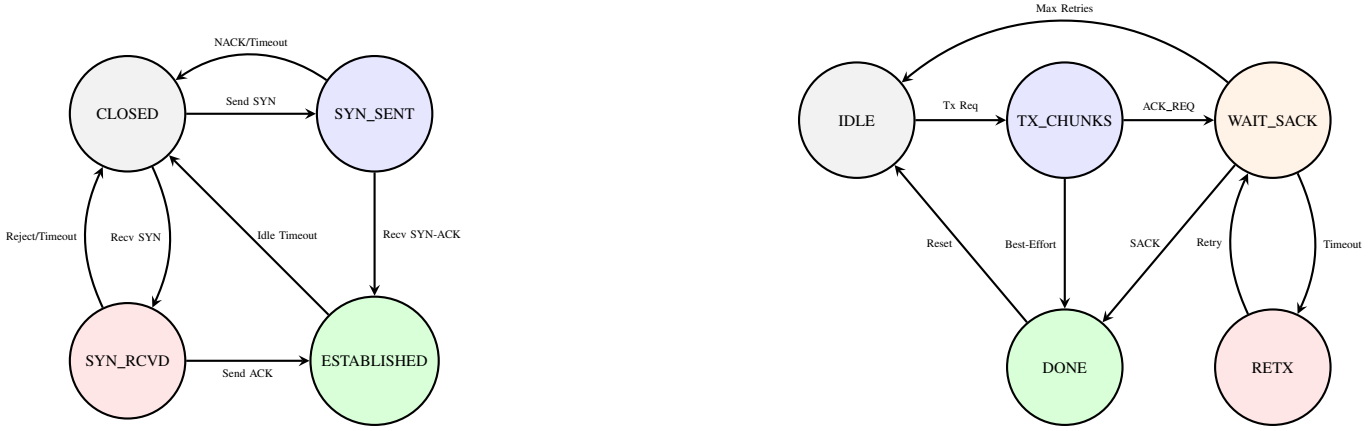
To ensure complete specification clarity and system-level reproducibility across different hardware platforms, Algorithm 1 and Algorithm 2 formally detail the transmitter-side segmentation pipeline and the receiver-side out-of-order reassembly engine, respectively.

F. Session Control and Handshake Protocol

To support stateful stream negotiation and protect resource-constrained receivers from buffer overflows, LAMPv2 introduces a 3WHS mechanism.

1) Three-Way Handshake Procedure: As illustrated in Figure 3(a), the connection lifecycle follows a three-step exchange:

1) **SYN**: The initiator sends a `SYN` control frame (`FLAG_CONN_REQ`) addressed to the target node's `dstAddr`, containing a 4-byte `SynMetadata` payload (`requestedPayloadSize`, `windowSize`, `timeoutMs`).
2) **SYN-ACK**: Upon receiving the `SYN`, the peer evaluates available memory and protocol version. If accepted, it responds with a `SYN-ACK` control frame (`FLAG_CONN_REQ | FLAG_CONN_ACK`) addressed back to the initiator. The receiver clamps



(a) Connection Session Lifecycle FSM (Stateful vs. Stateless)

(b) Data Transfer & SACK-ARQ FSM (Reliable vs. Unreliable)

Fig. 2. Finite State Machine (FSM) specifications for connection session lifecycle and data transfer modes.

Algorithm 1 LAMP Payload Segmentation and Framing (Transmitter)

Require: Payload P (size L), $dstAddr$, Max Chunk Size S, Mode M
Ensure: Sequence of frames $F_0, \ldots, F_{N-1}$
$N \leftarrow \lceil L/S \rceil$
if $N > 255$ **then**
  return ERR_PAYLOAD_TOO_LARGE
end if
$msgId \leftarrow$ GetNextMsgId()
for $i \leftarrow 0$ **to** $N-1$ **do**
  $chunkLen \leftarrow (i == N-1)?(L - i \cdot S) : S$
  $flags \leftarrow$ FLAG_NONE
  if $M \in \{\text{ReliableDatagram}, \text{ReliableStream}\}$ **and** $i == N-1$ **then**
    $flags \leftarrow flags \mid$ FLAG_ACK_REQ
  end if
  $hdr \leftarrow$ BuildHdr($srcAddr, dstAddr, msgId, N, i, chunkLen, flags$)
  $crc \leftarrow$ CalcCRC16($hdr, P[i \cdot S : i \cdot S + chunkLen]$)
  $F_i \leftarrow$ SerializeFrame($hdr, P[i \cdot S : i \cdot S + chunkLen], crc$)
  Transmit(F_i)
end for

Algorithm 2 Out-of-Order Reassembly & Session Tracking (Receiver)

Require: Received Frame F
Ensure: Reassembled Payload P or Drop Status
$(hdr, payload, crc) \leftarrow$ ParseFrame(F)
if ValidateCRC($hdr, payload, crc$) == FAIL **then**
  Drop(F); **return**
end if
if $hdr.dstAddr \neq myAddr$ **and** $hdr.dstAddr \neq$ 0xFF **then**
  Drop(F); **return**
end if
$session \leftarrow$ FindSession($hdr.srcAddr, hdr.msgId$)
if $session ==$ NULL **then**
  if AvailMem() $< hdr.totalChunks \cdot hdr.payloadSize$ **then**
    SendNack(hdr.srcAddr, BUSY); **return**
  end if
  $session \leftarrow$ AllocBuf($hdr.totalChunks, hdr.payloadSize$)
end if
$idx \leftarrow hdr.chunkIndex$
if IsBitSet($session.bitmap, idx$) **then**
  Drop(F) ▷ Duplicate chunk, ignore write
else
  WriteBuf($session.buffer[idx \cdot S], payload, hdr.payloadSize$)
  SetBit($session.bitmap, idx$)
  $session.count \leftarrow session.count + 1$
end if
if $hdr.flags$ & FLAG_ACK_REQ **then**
  SendSack($hdr.srcAddr, hdr.msgId, session.bitmap$)
end if
if $session.count == hdr.totalChunks$ **then**
  $P \leftarrow$ ExtractPayload($session.buffer$)
  DeliverToApp(P); FreeSession($session$)
end if

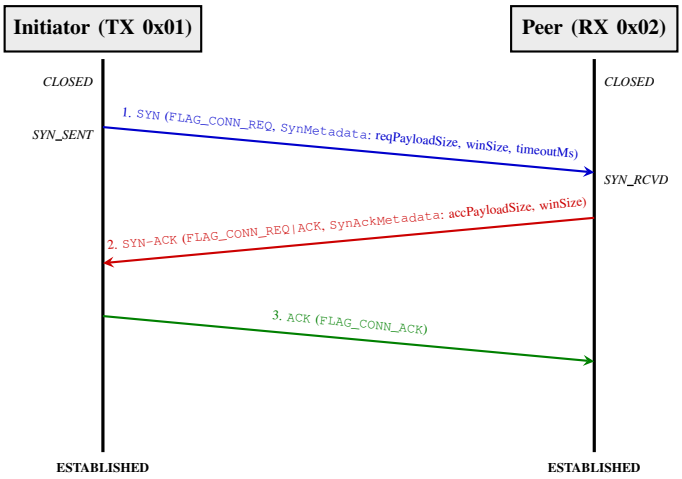
acceptedPayloadSize to its maximum buffer capability.

3) **ACK**: The initiator parses the SYN-ACK, adopts the negotiated payload size, and transmits a final ACK frame (FLAG_CONN_ACK). Both nodes transition their internal state machine to ESTABLISHED.

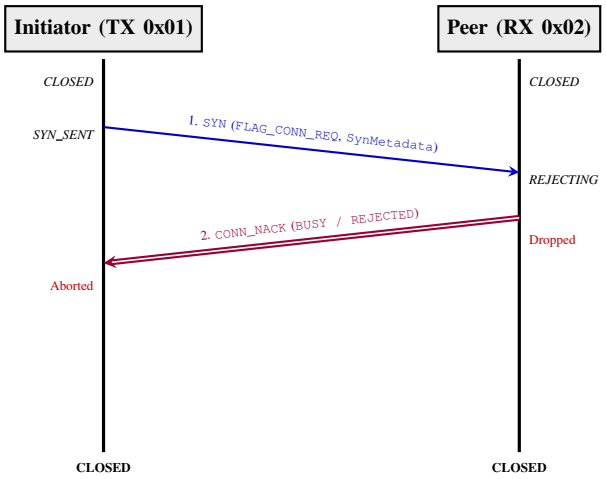
G. Concurrency and Fault Handling

To ensure protocol stability in multi-node and lossy environments, LAMP incorporates the following fault-handling mechanisms:

- *Simultaneous Connection Resolution*: If two nodes transmit SYN frames to each other simultaneously, a deterministic tie-breaking rule resolves the conflict: the node with the lower numerical NodeAddress yields and transitions back to CLOSED, while the node with the higher NodeAddress proceeds with session setup.
- *Receiver Busy Rejection*: If a receiver receives a SYN frame while its active reassembly session is already locked by another node, it responds with a CONN_NACK frame (FLAG_CONN_NACK) specifying a BUSY status code, causing the requester to back off.
- *Source Address Filtering*: Initiators in waitForSynAck and waitForSack validate incoming control frames against the expected srcAddr, ignoring packets from



(a) Successful 3-Way Handshake (3WHS) with Metadata Negotiation



(b) Connection Rejection (CONN_NACK) Failure Mode

Fig. 3. Sequence interaction flows for 3-Way Handshake (3WHS) connection setup with dynamic metadata negotiation and rejection handling.

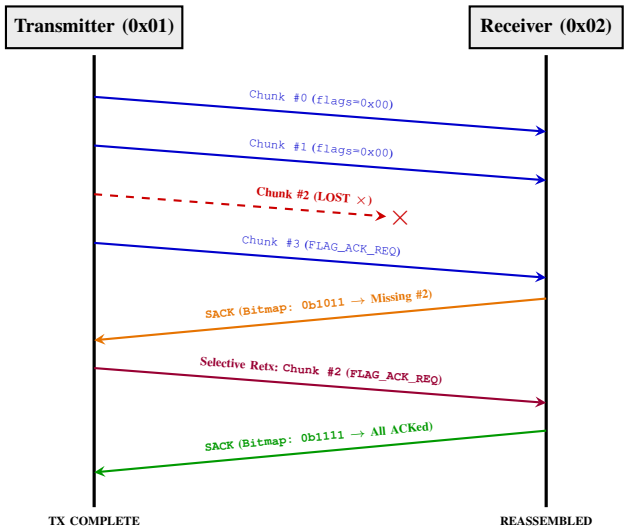


Fig. 4. End-to-end data streaming flow, frame loss detection, and SACK-driven selective retransmission interaction.

unrelated third-party nodes.

- *Incomplete Handshake Timeout*: If a SYN−ACK or ACK of a 3WHS is lost in transit, the receiver in SYN_RCVD or initiator in SYN_SENT automatically aborts and resets to CLOSED after an inactivity timeout (timeoutMs).

IV. Preliminary Proof of Concept

An initial prototype using commercial Ebyte E220-series modules validated the core segmentation concept. Field trials confirmed feasibility but exposed three limitations: (1) the modules exposed a proprietary network layer that prevented direct control of physical-layer parameters; (2) a large, unoptimized application header inflated per-chunk airtime; and (3) no acknowledgment or retransmission mechanism was available.

These findings motivated the redesign on Semtech SX126x-class transceivers via the RadioLib driver, validated on Heltec Wi-Fi LoRa© 32 V3 boards. The revised implementation—described in the following sections—is fully hardware-agnostic through the HAL interface and incorporates the 9-byte Logical Header, CRC-16 integrity checks, and the SACK-based retransmission mechanism.

V. Hardware Level Implementation

To validate the proposed protocol, a complete implementation of the LAMP library was developed in C++17. The design is modular, separating the transport state machine from physical hardware dependencies.

A. Hardware Abstraction Layer (HAL)

To keep the protocol core hardware-agnostic, a clean HAL interface (RadioLibHal) was implemented. On the physical nodes, the concrete class EspHal wraps the ESP32-S3 hardware resources:

- **Serial Peripheral Interface (SPI) Communication**: Manages registers configuration and FIFO buffers transfer for the Semtech SX1262 transceiver.
- **General-Purpose Input/Output (GPIO) Interrupts**: Maps physical pins, including Chip Select (NSS), Reset (RST), Busy status indicator (BUSY), and the hardware interrupt pin (DIO1) used to trigger RX/TX completion interrupts.
- **Timing Primitives**: Implements millisecond counters (millis()) and microsecond/millisecond delays (delay()).

B. Host-Based Unit Testing

Following a TDD methodology, the protocol core (e.g., packet parsing, reassembler sessions, and CRC checksums) was validated in isolation on a desktop host (Linux/macOS). The tests compile and run natively using the **Unity** testing framework integrated into PlatformIO:

- **Integrity Tests**: Confirmed the Modbus CRC-16 computation on headers and partial/padded payloads.

- **Reassembly State Machine**: Validated reassembly handling of out-of-order and duplicate chunks.
- **Stale Session Pruning**: Confirmed that incomplete reassembly buffers are correctly pruned after exceeding the timeout threshold.

C. Physical Benchmarking Setup

Physical tests were conducted using two Heltec Wi-Fi LoRa© 32 V3 development boards. The transmitter node was connected to a console runner to sweep payload sizes, while the receiver node operated as a standalone listener. The transceivers were configured with the following parameters:

- **Carrier Frequency**: 869.525 MHz
- **Spreading Factor (SF)**: 7
- **Bandwidth (BW)**: 500.0 kHz
- **Coding Rate (CR)**: 4/5
- **Output Power**: 10 dBm
- **Preamble Length**: 8 symbols

1) Regulatory Compliance and Reproducibility: It is important to clarify the relationship between the LAMP transport protocol and RF regulatory constraints. LAMP operates strictly at the *transport layer* and is entirely agnostic to duty-cycle regulations, carrier frequency, and RF power limits; compliance with these constraints is the responsibility of the MAC/PHY layer or the integrating application.

For the experimental validation reported in this work, the 869.400–869.650 MHz sub-band was selected [6]. Under ETSI EN 300 220-1, this sub-band permits a maximum duty cycle of 10% and an output power of up to 500 mW (27 dBm), providing ample margin for the multi-chunk burst transfers required by the LAMP reliable delivery modes without incurring the restrictive 1% hourly airtime cap of the primary 868 MHz sub-band.

Additionally, in the reference implementation validated on Heltec V3 hardware, the optional MAC-layer helpers provided by the LAMP library—namely SX1262-native Channel Activity Detection (CAD) with randomized exponential backoff (CSMA-CA) and duty-cycle pacing—were enabled to prevent collisions and further reduce channel occupancy. These features are opt-in and disabled by default; they are not part of the LAMP transport specification but are offered as convenience primitives for integrators targeting shared ISM bands. All experimental results are therefore fully reproducible in any deployment targeting the 869.4–869.65 MHz sub-band with the same radio configuration.

VI. Proposed Experimental Methodology

To validate the reliability, latency, and throughput of the LAMP protocol, we outline a series of physical field experiments to be conducted under representative operational scenarios.

A. Experimental Parameters

The validation strategy follows established LPWAN characterization methodologies [7] and will sweep the following parameters:

- **Transmission Distance and Spatial Geometry**: Tests will be carried out at physical distances of 1, 10, 100, 250, 500, 1000, 2500, and 5000 meters. To account for spatial fading, antenna polarization discrepancies, and localized multi-path fading, testing in the short range (from 0 to 100 meters) will utilize a circular spatial sampling strategy, gathering data points at multiple angular offsets around the transmitting antenna. For long-range setups (greater than 100 meters) where circular navigation is impractical, data collection will follow a linear path.
- **Propagation Conditions**: All distance steps will be benchmarked under both Line-of-Sight (LoS) and Non-Line-of-Sight (NLoS) environments to test protocol performance under varying noise and packet corruption conditions [7].
- **Payload Sequence Size**: Payloads will scale from 1 chunk (representing the boundary condition) up to 100 chunks (approximately 24.6 KB total data size).
- **Radio Configuration**: Nodes will maintain standard settings (SF7, BW 500 kHz, CR 4/5, 10 dBm TX power) using the integrated Semtech transceivers.

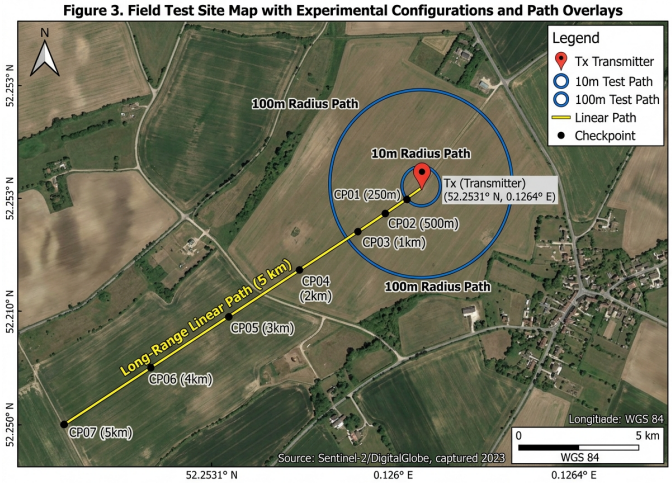


Fig. 5. Satellite view of the experimental testbed showing the transmitter node location, the circular spatial sampling points ($\leq$ 100 meters), and the linear range checkpoints ($>$ 100 meters) under LoS and NLoS profiles.

B. Evaluation Metrics

For each combination of distance, environment, and payload size, 15 independent iterations will be performed. The following metrics will be collected and analyzed:

1) **Packet Delivery Ratio (PDR)**: The percentage of successfully reconstructed payloads at the receiver.
2) **End-to-End Latency**: The total duration elapsed from the initiation of split-packet serialization at the transmitter to the final payload reassembly callback at the receiver.
3) **Throughput (Goodput)**: The effective rate of application-level payload data delivered, calculated as $\text{PayloadSize (bits)}/\text{Latency (seconds)}$.
4) **Retransmission Overhead**: The count of extra chunks transmitted in reliable mode compared to the baseline

case, representing the cost of SACK-driven error correction.

These benchmarks will allow us to evaluate the efficiency of the SACK retransmission handshake compared to best-effort unreliable streaming in lossy physical channels. The empty template for recording these empirical results is structured in Table VI.

VII. Conclusion and Architectural Roadmap

This paper presented the design, implementation, and hardware validation of LAMP, a lightweight and reliable transport-layer architecture optimized for transmitting segmented large payloads over point-to-point LoRa© links. By replacing traditional Stop-and-Wait per-packet acknowledgments with selective retransmissions requested via cumulative binary bitmaps, and eliminating redundant SOM/EOM markers in favor of implicit index derivation, the protocol achieves high reliability under lossy conditions with minimal airtime overhead and duty-cycle footprint.

A. Architectural Roadmap

To guide the long-term progression of LAMP from point-to-point data links to decentralized multiagent mesh networks, we define a strategic three-level architectural roadmap:

- *Point-to-Point Transport (Current Stack)*: The core protocol stack established in this work. It features 8-bit source and destination node addressing (`srcAddr`, `dstAddr`), dynamic 9-byte header packing, four operational modes (unreliable best-effort datagrams to stateful SACK-ARQ streams), a 3WHS for parameter negotiation, and hardware-validated edge-case resolution (BUSY rejection, address filtering, and simultaneous SYN tie-breaking).
- *Local Multi-Node Networking*: Extending the link layer to support local multi-node clusters sharing the same RF channel. Key features will include:
  - *Broadcast and Multicast Messaging*: Utilizing `0xFF` for unacknowledged broadcast telemetry and group addressing for multiagent synchronization.
  - *MAC-Layer Channel Access*: Standardizing the optional CAD/CSMA-CA and AFA primitives currently provided as opt-in library helpers into a formally specified MAC sub-layer, enabling full *Polite Spectrum Access* (Listen-Before-Talk + Adaptive Frequency Agility) as defined in ETSI EN 300 220-1 [6] for deployments requiring compliance beyond the 10% duty-cycle sub-band. This removes the duty-cycle percentage constraint entirely, replacing it with per-packet burst duration limits and minimum backoff windows, and supports Time Division Multiplexing (TDM)-scheduled channels for dense deployments [12].
  - *Adaptive Data Rate (Air Data Rate (ADR))*: Dynamically tuning Spreading Factor (SF7–SF12) and Coding Rate (CR) based on RSSI/SNR metrics embedded in SACK bitmaps.
- *Multi-Hop Mesh Extension*: Scaling the architecture to autonomous multi-hop networks, such as UAV swarms, distributed sensor meshes, and relay networks [10], [11]:
  - *Mesh Routing Protocol*: Integrating dynamic routing (e.g., AODV-Light or RPL adaptation) with hop-by-hop vs. end-to-end SACK reliability options to limit retransmission overhead over multi-hop links [11].
  - *IPv6 Network Layer Integration*: Leveraging the 8-bit Node ID mapping to establish full IPv6 link-local capability (`fe80::/64`), connecting micro-drones directly to global IP infrastructure.
  - *TDMA / Scheduled Time-Slotted Access*: Incorporating microsecond time synchronization for collision-free multiagent payload transfers in high-density networks.

Declarations

Data Availability Statement

All firmware implementations, C++17 library source code, host unit testing suites, and benchmarking scripts are available in the public project repository.

Ethical Statement and AI Tool Usage Disclosure

This study did not involve human participants or animal subjects. AI assistant tools were utilized strictly for code formatting, micro-typography auditing, and manuscript style calibration in compliance with IEEE publication ethics guidelines.

Conflict of Interest Statement

The authors declare that they have no competing financial or personal interests that could influence the work reported in this paper.

References

[1] M. Radeta, J. Pestana, P. Abreu, R. Freitas, F. Silva, D. Vieira, R. Prieto, M. Fernandez, F. Alves, T. Dellinger, S. Neves, and E. Delory, "Triton—open telemetry and location estimation for marine monitoring based on iot and lora," *IEEE Journal of Oceanic Engineering*, vol. 50, no. 2, pp. 1244–1258, 2025.

[2] T. Elshabrawy and J. Al-Ali, "Lora technology demystified: From link behavior to cell capacity," *IEEE Transactions on Wireless Communications*, vol. 17, no. 10, pp. 6611–6621, 2018.

[3] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melia-Segui, and T. Watteyne, "Understanding the limits of lorawan implementations," *IEEE Communications Magazine*, vol. 55, no. 9, pp. 34–40, 2017.

[4] S. De, H. M. Jalajamony, S. Adhinarayanan, S. Joshi, H. Upadhyay, and R. Fernandez, "Multimedia transmission over lora networks for iot applications: A survey of strategies, deployments, and open challenges," *Sensors*, vol. 25, no. 23, p. 7128, 2025.

[5] P. J. Marcelis, V. S. Rao, and R. V. Prasad, "Dare: Data recovery through application layer coding for lorawan," in *Proceedings of the 2017 IEEE/ACM Second International Conference on Internet-of-Things Design and Implementation (IoTDI)*, 2017.

[6] *Short Range Devices (SRD) operating in the frequency range 25 MHz to 1 000 MHz; Part 1: Technical characteristics and methods of measurement*, European Telecommunications Standards Institute (ETSI) Std. ETSI EN 300 220-1 V3.1.1, 2017.

[7] M. Cattani, C. A. Boano, and K. Römer, "An experimental evaluation of the reliability of lora long-range low-power wireless communication," *Journal of Sensor and Actuator Networks*, vol. 6, no. 2, p. 7, 2017.

TABLE VI
LAMP PLANNED EXPERIMENTAL BENCHMARKING RESULTS TEMPLATE

Distance (m)	Path Type	Payload Chunks	PDR (%)		Avg Latency (ms)		Avg Goodput (Kbps)		Retransmission Overhead	
			Case A	Case B	Case A	Case B	Case A	Case B	Chunks Sent	Retries
1	LoS	1–100								
	NLoS	1–100								
10	LoS	1–100								
	NLoS	1–100								
100	LoS	1–100								
	NLoS	1–100								
250	LoS	1–100								
	NLoS	1–100								
500	LoS	1–100								
	NLoS	1–100								
1000	LoS	1–100								
	NLoS	1–100								
2500	LoS	1–100								
	NLoS	1–100								
5000	LoS	1–100								
	NLoS	1–100								

[8] M. Bor, U. Roedig, T. Voigt, and J. M. Alonso, "Do lora low-power wide-area networks scale?" in *Proceedings of the 19th ACM International Conference on Modeling, Analysis and Simulation of Wireless and Mobile Systems (MSWiM)*, 2016, pp. 59–66.

[9] T. Chen, D. Eager, and D. Makaroff, "Efficient image transmission using lora technology in agricultural monitoring iot systems," in *2019 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, 2019, pp. 937–944.

[10] J. Kim, J. Jenkins, J. Seol, and M. Kwak, "Extending data transmission in the multi-hop lora network," *Issues in Information Systems*, vol. 23, no. 3, pp. 292–304, 2022.

[11] A. W.-L. Wong, S. L. Goh, M. K. Hasan, and S. Fattah, "Multi-hop and mesh for lora networks: Recent advancements, issues, and recommended applications," *ACM Computing Surveys*, vol. 56, no. 6, pp. 1–43, 2024.

[12] C.-C. Wei, J.-K. Huang, C.-C. Chang, and K.-C. Chang, "The development of lora image transmission based on time division multiplexing," in *2020 International Symposium on Computer, Consumer and Control (IS3C)*, 2020, pp. 323–326.

[13] A. Calabrò, E. Marchetti, and M. T. Paratore, "Evaluating use of arq strategies in communication protocols for search and rescue," in *Proceedings of the 21st International Conference on Web Information Systems and Technologies (WEBIST)*. SCITEPRESS, 2025, pp. 59–70.

[14] S. Corporation, "Lora modem designer's guide," Tech. Rep. Application Note AN1200.13, 2015.